



UNIVERSIDAD DE LAS FUERZAS ARMADAS
ESPE

CONSTRUCCIÓN Y EVOLUCIÓN DEL SOFTWARE

DEPARTAMENTO DE CIENCIAS DE LA COMPUTACIÓN

Evolución del Software, Reingeniería y Sistemas Legados

Estudiante:

Ednan Josué Merino Calderón

Docente:

Ing. Ángel Marcelo Rea Guamán Ph.D

Índice general

1. Antecedentes	2
2. Objetivos	3
2.1. Objetivo General	3
2.2. Objetivos Específicos	3
3. Evolución del Software	4
3.1. Definición y Naturaleza del Cambio	4
3.2. El Modelo por Etapas (Staged Model)	4
3.3. Tipos de Evolución de Software	5
4. Mecánica de Cambio y Sistemas Legados	6
4.1. Phased Model of Software Change (PMSC)	6
4.2. Sistemas Legados y el Fenómeno del Decaimiento	7
4.3. Ingeniería Inversa	8
4.4. Reingeniería de Software	8
5. Conclusiones	10

1. Antecedentes

La ingeniería de software ha experimentado un cambio de paradigma desde la década de 1970, pasando de un enfoque centrado en la creación de sistemas desde cero hacia una disciplina donde el cambio y la adaptación son la norma. Históricamente, se consideraba que el desarrollo terminaba con la entrega del producto; sin embargo, la realidad industrial reveló que la mayor parte del presupuesto y el esfuerzo técnico se destinan a la fase posterior a la entrega.

Este fenómeno fue formalizado por Meir Lehman a través de sus Leyes de Evolución del Software, las cuales establecen que un sistema que opera en un entorno real debe evolucionar continuamente o, de lo contrario, se vuelve progresivamente menos útil debido a la pérdida de alineación con el contexto del negocio. En las últimas décadas, el auge del desarrollo evolutivo de software (ESD, por sus siglas en inglés), que incluye procesos ágiles, iterativos y de código abierto, ha desplazado el núcleo de la ingeniería de software hacia la etapa de evolución. Actualmente, se estima que para sistemas exitosos, una cantidad abrumadora de tiempo y recursos se invierte en esta etapa central del ciclo de vida.

2. Objetivos

2.1. Objetivo General

Analizar de manera técnica y sistemática los fundamentos de la evolución del software, profundizando en los modelos de ciclo de vida por etapas y los procesos de transformación de sistemas legados para comprender su preservación de valor en el tiempo.

2.2. Objetivos Específicos

- Definir la evolución del software diferenciándola del mantenimiento tradicional mediante el análisis del modelo por etapas propuesto por Rajlich.
- Desglosar las fases del Phased Model of Software Change (PMS) para entender la mecánica interna de las modificaciones en sistemas complejos.
- Investigar las causas del decaimiento del código y el rol de la ingeniería inversa y la reingeniería en la mitigación de la deuda técnica de los sistemas legados.

3. Evolución del Software

3.1. Definición y Naturaleza del Cambio

La evolución del software se define como el proceso dinámico de cambio constante provocado por la volatilidad de los requisitos, las tecnologías y el conocimiento de los interesados. A diferencia del mantenimiento, que a menudo se percibe como una actividad de reparación secundaria, la evolución constituye el centro de atención del desarrollo moderno. En este contexto, el software se percibe como un ente dinámico que debe adaptarse para sobrevivir a la entropía de software, un estado donde la complejidad crece de forma descontrolada si no se aplican medidas de refactorización y reingeniería constantes.

3.2. El Modelo por Etapas (Staged Model)

De acuerdo con Rajlich, el ciclo de vida del software no es una línea uniforme, sino que se divide en cinco etapas críticas que definen su capacidad de evolución:

- **Desarrollo Inicial:** Producción de la primera versión desde cero, estableciendo decisiones arquitectónicas fundamentales que son costosas de revertir posteriormente.
- **Evolución:** La etapa central donde se agregan nuevas funcionalidades y se corrigen errores de diseño para reaccionar a la volatilidad del entorno.
- **Servicio (Mantenimiento):** El sistema deja de recibir cambios mayores. Los programadores solo realizan parches menores (servicing patches) para mantenerlo usable. Aquí el sistema comienza a ser etiquetado como "legado".

- **Fase de Retiro (Phase-out):** No se realizan más reparaciones, aunque el sistema sigue en uso hasta su sustitución.
- **Cierre (Close-down):** El software es retirado completamente de producción.

3.3. Tipos de Evolución de Software

La evolución se manifiesta a través de diversas categorías de mantenimiento, cada una con un impacto distinto en la arquitectura:

- **Evolución Correctiva:** Enfocada en la resolución de fallas que impiden el cumplimiento de los requisitos especificados.
- **Evolución Adaptativa:** Necesaria cuando el entorno operativo (hardware, sistema operativo, librerías) cambia, exigiendo que el software se ajuste para mantener su funcionalidad original.
- **Evolución Perfectiva:** Implementación de mejoras solicitadas por los usuarios que añaden valor competitivo o eficiencia al sistema.
- **Evolución Preventiva:** Acciones proactivas como la refactorización, orientadas a mejorar la estructura interna y facilitar cambios futuros, reduciendo la probabilidad de errores.

4. Mecánica de Cambio y Sistemas Legados

4.1. Phased Model of Software Change (PMSC)

Para que un ingeniero realice cambios efectivos en un sistema en evolución, debe seguir un proceso sistemático que minimice la introducción de errores. El modelo PMSC propone las siguientes fases técnicas:

1. **Iniciación:** Análisis y priorización de los requisitos de cambio.
2. **Localización de Conceptos:** Búsqueda del fragmento de código específico (extensiones) que implementa un concepto de dominio. Es crítica en sistemas grandes donde los desarrolladores operan bajo una comprensión "según sea necesario" (as-needed).
3. **Análisis de Impacto:** Determinación de la estrategia y la extensión total de los componentes afectados por el cambio propuesto.
4. **Prefactorización:** Reestructuración del código existente para facilitar la implementación del nuevo cambio.
5. **Actualización:** Implementación física del cambio y propagación a través de las dependencias.
6. **Postfactorización:** Limpieza técnica y eliminación de deuda técnica introducida durante la actualización.
7. **Conclusión:** Verificación, commit en el sistema de control de versiones y creación de una nueva línea base.

Phased Model of Software Change (PMSC)

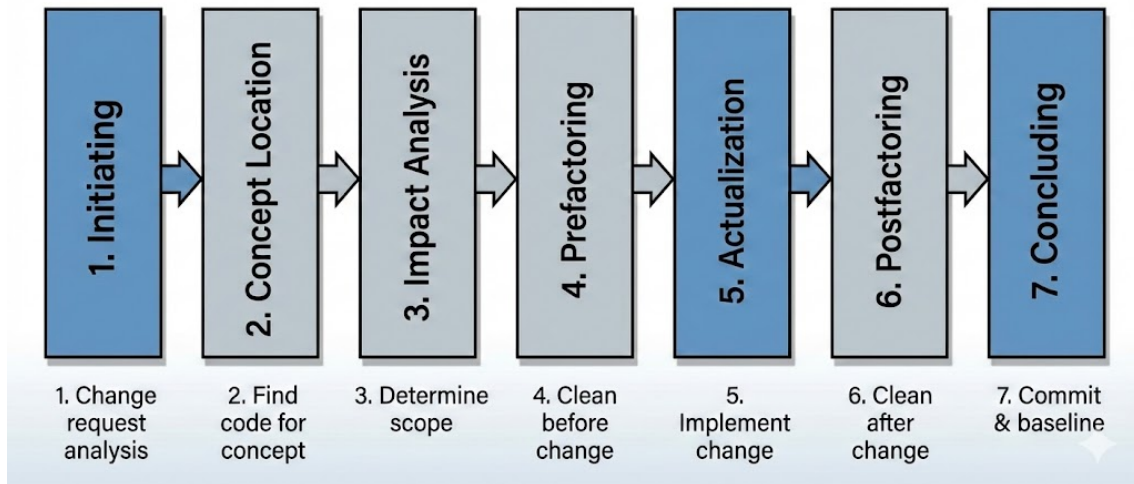


Figura 4.1: Phased Model of Software Change (PMSC)

4.2. Sistemas Legados y el Fenómeno del Decaimiento

Un Sistema Legado no se define simplemente por su edad cronológica, sino por su estado dentro de la organización. Es un sistema informático que ha sido desarrollado en el pasado, a menudo con tecnologías ahora obsoletas, pero que continúa desempeñando una función crítica para el negocio. La paradoja de los sistemas legados radica en que son simultáneamente un activo indispensable y un pasivo técnico creciente. Su reemplazo suele ser inviable debido a los altos costos, los riesgos operativos y, frecuentemente, porque contienen reglas de negocio vitales que no están documentadas en ningún otro lugar más que en el propio código fuente [3].

El principal desafío técnico que presentan estos sistemas es el Decaimiento del Código (Code Decay). Este fenómeno describe la degradación progresiva de la estructura y calidad interna del software como consecuencia de múltiples y continuos cambios a lo largo del tiempo. A medida que se aplican parches y modificaciones sin una refactorización adecuada, la arquitectura original se erosiona, la coherencia se pierde y la complejidad ciclomática aumenta. Este proceso incrementa exponencialmente la dificultad para realizar nuevos cambios, ya que la comprensión del sistema se vuelve difusa incluso para los ingenieros experimentados. El decaimiento transfor-

ma la evolución ágil en un mantenimiento correctivo costoso y propenso a errores, marcando la transición del sistema hacia un estado de rigidez.

4.3. Ingeniería Inversa

Ante la pérdida de documentación actualizada y el conocimiento tácito de los desarrolladores originales, la Ingeniería Inversa emerge como la disciplina fundamental para recuperar el control sobre un sistema legado. Definida formalmente por Chikofsky y Cross, es el proceso de analizar un sistema para identificar los componentes del mismo y las interrelaciones entre ellos, con el fin de crear representaciones del sistema en otro nivel de abstracción, generalmente más alto [5].

Es crucial entender que la ingeniería inversa no implica la modificación del sistema, ni es sinónimo de "descompilar". Su objetivo principal es la comprensión. A través de técnicas estáticas (análisis del código fuente sin ejecutarlo) y dinámicas (análisis del comportamiento del sistema en tiempo de ejecución), los ingenieros buscan:

- Recuperar la arquitectura de diseño perdida.
- Redocumentar la funcionalidad y las reglas de negocio.
- Identificar componentes reutilizables.
- Detectar áreas de alto acoplamiento y baja cohesión que son candidatas a reestructuración.

La ingeniería inversa es, por tanto, una actividad de diagnóstico esencial que precede a cualquier intento serio de reingeniería.

4.4. Reingeniería de Software

La Reingeniería de Software es el proceso sistemático de transformación de un sistema existente para mejorarlo, ya sea en su estructura interna, en sus características funcionales o en ambas, manteniendo la continuidad del servicio. A diferencia de la ingeniería inversa que solo analiza, la reingeniería implica la modificación activa del código y la arquitectura.

El objetivo central de la reingeniería es combatir el decaimiento del código y extender la vida útil del sistema, transformándolo de un estado "legado" difícil de mantener a un estado más evolutivo y adaptable a las nuevas necesidades del negocio. Un proceso típico de reingeniería incluye las siguientes fases:

1. **Ingeniería Inversa:** Para comprender el estado actual del sistema.
2. **Reestructuración del Programa:** Modificación de la estructura interna del código (refactorización a gran escala) para mejorar su lógica y claridad, sin alterar su comportamiento externo.
3. **Reestructuración de Datos:** Análisis y rediseño de las estructuras de datos y esquemas de base de datos para mejorar su integridad y rendimiento.
4. **Ingeniería Directa (Forward Engineering):** La reconstrucción o implementación de partes del sistema utilizando tecnologías y prácticas modernas, basadas en la nueva estructura definida.

La reingeniería es una inversión estratégica que busca reducir los costos de mantenimiento a largo plazo y recuperar la agilidad técnica perdida [2].

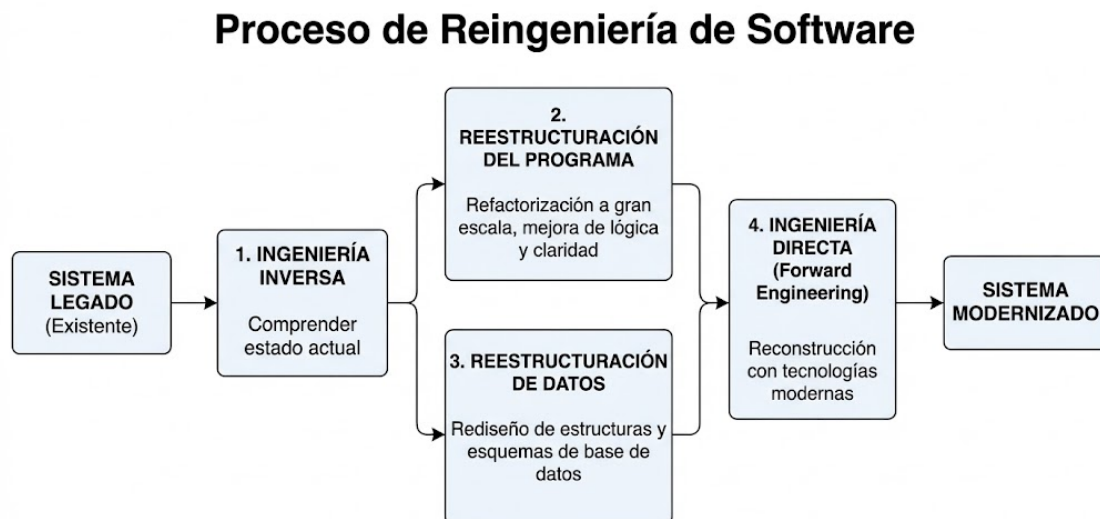


Figura 4.2: Reingeniería del Software

5. Conclusiones

- El análisis del modelo por etapas de Rajlich permite redefinir la evolución del software no como una simple fase de corrección de errores, sino como el estado predominante del ciclo de vida donde el sistema mantiene su vitalidad mediante adaptaciones arquitectónicas significativas. Se establece una clara distinción técnica con la fase de servicio (mantenimiento tradicional), la cual se limita a parches menores y marca el inicio de la obsolescencia del sistema.
- El Modelo de Cambio por Fases (PMSC) revela que la modificación de software complejo es un proceso ingenieril sistemático y no una mera actividad de codificación. Fases analíticas como la localización de conceptos y el análisis de impacto son determinantes para el éxito del cambio, requiriendo una alta carga cognitiva para comprender la dispersión de la funcionalidad en el código y prever las consecuencias laterales de cualquier modificación.
- El decaimiento del código en sistemas legados es una consecuencia directa de la acumulación de deuda técnica y la pérdida de conocimiento arquitectónico a lo largo del tiempo. Frente a este desafío, la ingeniería inversa actúa como la herramienta diagnóstica necesaria para recuperar la comprensión del diseño perdido, mientras que la reingeniería constituye el proceso terapéutico para reestructurar el sistema, mitigar el decaimiento y restaurar su capacidad de evolución.

Bibliografía

- [1] Rajlich, V. (2014). *Software evolution and maintenance*. Future of Software Engineering Proceedings (FOSE), 36th International Conference on Software Engineering, 133-144. doi:10.1145/2593882.2593893
- [2] Pressman, R. S., & Maxim, B. R. (2020). *Ingeniería del software: Un enfoque práctico* (9a ed.). McGraw-Hill Education.
- [3] Sommerville, I. (2016). *Ingeniería del Software* (10a ed.). Pearson Educación.
- [4] Lehman, M. M., & Belady, L. A. (1985). *Program Evolution: Processes of Software Change*. Academic Press Professional, Inc.
- [5] Chikofsky, E. J., & Cross, J. H. (1990). *Reverse Engineering and Design Recovery: A Taxonomy*. IEEE Software, 7(1), 13-17.
- [6] Mens, T., & Demeyer, S. (2008). *Software Evolution*. Springer Science & Business Media.